

Coleções em Java

ArrayList, HashSet e HashMap

Linguagem de Programação II

Objetivos da aula

- Entender por que usamos coleções em Java.
- Comparar vetores com coleções.
- Usar `ArrayList` para armazenar listas.
- Usar `HashSet` para evitar repetição.
- Usar `HashMap` para chave e valor.
- Manter atributos encapsulados com `private`.
- Inicializar coleções no construtor.
- Criar métodos para adicionar, remover, buscar e listar.

Por que estudar coleções?

Problemas com vetores

- Tamanho **fixo** na criação.
- Nem sempre sabemos quantos elementos serão cadastrados.
- Gerenciar espaço variável exige muito código manual.

Coleções resolvem isso

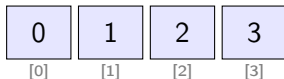
- Crescem **automaticamente**.
- Métodos prontos: add, remove, get, size...
- Código mais simples e seguro.

Ideia central

Coleções são estruturas prontas do Java para armazenar vários dados ou objetos de forma dinâmica.

Vetor × Coleção — comparação visual

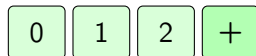
VETOR



tamanho fixo
sem métodos prontos

VS

COLEÇÃO



cresce quando necessário
métodos prontos

As três coleções que vamos estudar

ArrayList

Lista de elementos com **posição (índice)**.

Mantém ordem, permite repetição.

Ex: lista de alunos de uma turma.

HashSet

Conjunto **sem repetição**.

Não garante ordem dos elementos.

Ex: matrículas já cadastradas.

HashMap

Estrutura de **chave → valor**.

Busca instantânea pela chave.

Ex: produto localizado pelo código.

Imports necessários

```
import java.util.ArrayList;  
import java.util.HashSet;  
import java.util.HashMap;
```

ArrayList

Lista de elementos com posição (índice). Mantém ordem e permite repetição.

HashSet

Conjunto sem repetição. Não garante ordem dos elementos.

HashMap

Pares de chave e valor. Cada chave é única e aponta para um valor.

Atenção

Sem o import correto o Java não encontra a classe e o programa não compila.

Atenção: uma classe pública por arquivo

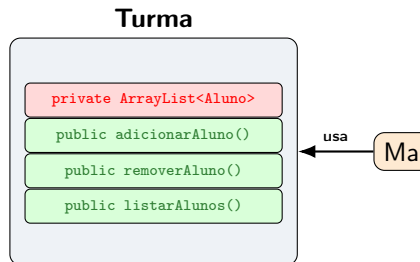
Regra importante

Em Java, cada classe `public` deve ficar em um arquivo separado com o mesmo nome da classe.

- `public class Aluno` → `Aluno.java`
- `public class Turma` → `Turma.java`
- `public class Produto` → `Produto.java`
- `public class Main` → `Main.java`

Encapsulamento com coleções

Mesmo usando coleções, os atributos continuam sendo **privados**.



Regras

- A coleção deve ser **private**.
- Inicializar no construtor (**public**).
- Acesso apenas por métodos públicos.
- O **main** nunca toca na coleção diretamente.

Inicialização correta no construtor

```
import java.util.ArrayList;

public class Turma {

    private ArrayList<Aluno> alunos;    // atributo privado

    public Turma() {
        alunos = new ArrayList<>();    // lista nasce vazia
    }
}
```

1. Atributo private

Ninguém de fora acessa a lista diretamente.

2. Construtor public

Garante que a lista é criada antes de ser usada.

3. Lista vazia

`new ArrayList<>()`
começa sem elementos.

O problema: lista sem tipo definido

Sem Generics (forma errada)

```
ArrayList alunos = new ArrayList();

alunos.add("Maria");
alunos.add(123);    // aceita int!
alunos.add(true);   // aceita boolean
!
```

- A lista aceita **qualquer** tipo.
- Erros aparecem só em **execução**.
- Recuperar dados exige *cast* manual — arriscado.

Com Generics (forma correta)

```
ArrayList<Aluno> alunos = new
    ArrayList<>();

alunos.add(new Aluno(1001, "Maria"))
    ; // ok
alunos.add("Maria"); // ERRO de
    compilacao
```

- Só aceita objetos **Aluno**.
- Erros detectados na **compilação**.
- Sem *cast* ao recuperar elementos.

O que são Generics?

Ideia simples

Generics indicam qual tipo de dado uma coleção pode guardar, usando `<` `>`.

`ArrayList`

`<Aluno>`

a coleção

o tipo guardião

Sem Generics

- Aceita qualquer tipo
- Erros em tempo de execução
- *Cast* obrigatório ao ler

Com Generics

- Só aceita o tipo declarado
- Erros na compilação
- Sem *cast* ao ler

Generics com diferentes tipos

```
ArrayList<String>  nomes      = new ArrayList<>();  
ArrayList<Integer> numeros    = new ArrayList<>();  
ArrayList<Aluno>   alunos     = new ArrayList<>();
```

Regra prática

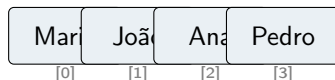
Sempre que soubermos o tipo dos elementos, devemos usar Generics. O Java protege a lista automaticamente.

Generics só aceitam tipos por referência

Não podemos escrever `ArrayList<int>`. Usamos `ArrayList<Integer>` (classe wrapper).

ArrayList — características

ArrayList<Aluno>



- Representa uma **lista** de elementos.
- Mantém a **ordem** de inserção.
- **Permite** elementos repetidos.
- Permite acessar elementos **por posição**.
- O índice começa em **zero**.
- Cresce **automaticamente**.

Quando usar?

Quando quiser armazenar uma sequência de elementos, com posição definida.

Criando um ArrayList

```
import java.util.ArrayList;  
  
ArrayList<String> nomes = new ArrayList<>();  
  
nomes.add("Maria");  
nomes.add("Joao");  
nomes.add("Ana");
```

ArrayList<String>

Tipo dos elementos
entre < >.

new ArrayList<>()

Cria a lista vazia. O <
> pode ficar vazio aqui.

.add(x)

Adiciona o elemento x
ao final da lista.

Métodos essenciais do ArrayList

```
nomes.add("Maria");           // adiciona ao final
nomes.add(0, "Bia");          // insere na posicao 0
nomes.get(0);                 // pega pelo indice
nomes.size();                 // quantidade de elementos
nomes.remove(0);              // remove pelo indice
nomes.remove("Maria");        // remove pelo valor
nomes.contains("Ana");        // existe na lista? true/false
nomes.isEmpty();              // lista esta vazia? true/false
nomes.clear();                // remove todos os elementos
```

Dica

remove(0) remove pela **posição**; remove("Maria") remove pelo **valor**.
Cuidado ao usar com Integer!

Percorrendo um ArrayList

Com índice (use quando precisar da posição):

```
for (int i = 0; i < nomes.size(); i++) {  
    System.out.println(nomes.get(i));  
}
```

Com for-each (mais simples — preferido para leitura):

```
for (String nome : nomes) {  
    System.out.println(nome);  
}
```

Laço com índice

Use quando precisar da **posição** ou quiser modificar elementos específicos.

For-each

Use para **leitura** simples. Mais legível e menos propenso a erros.

Classe Aluno encapsulada

```
public class Aluno {  
  
    private int matricula;  
    private String nome;  
  
    public Aluno(int matricula, String nome) {  
        this.matricula = matricula;  
        this.nome = nome;  
    }  
  
    public int getMatricula() {  
        return matricula;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

Lembre-se

Salvar em Aluno.java. Atributos private, acesso pelos *getters* public.

Classe Turma com ArrayList

```
import java.util.ArrayList;

public class Turma {

    private String codigoDisciplina;
    private ArrayList<Aluno> alunos;    // (1) private

    public Turma(String codigoDisciplina) {
        this.codigoDisciplina = codigoDisciplina;
        alunos = new ArrayList<>();    // (2) inicializa no
        construtor
    }

    public void adicionarAluno(Aluno aluno) {
        alunos.add(aluno);    // (3) acesso pelo metodo
    }

    public int getQuantidadeAlunos() {
        return alunos.size();
    }
}
```

Buscando aluno pela matrícula

```
public Aluno buscarAlunoPorMatricula(int matricula) {  
    for (Aluno alunoAtual : alunos) {  
        if (alunoAtual.getMatricula() == matricula) {  
            return alunoAtual;    // encontrou: retorna  
        }  
    }  
    return null;    // nao encontrado  
}
```

1. Percorrer

for-each visita cada aluno da lista.

2. Comparar

getMatricula() acessa o atributo de forma encapsulada.

3. Retornar

Retorna o aluno se achar; null se não achar.

Removendo aluno pela matrícula

```
public boolean removerAlunoPorMatricula(int matricula) {  
    Aluno alunoEncontrado = buscarAlunoPorMatricula(matricula);  
  
    if (alunoEncontrado != null) {  
        alunos.remove(alunoEncontrado);  
        return true;        // removido com sucesso  
    }  
  
    return false;           // nao encontrado  
}
```

Estratégia: buscar e remover

Reutilizamos

`buscarAlunoPorMatricula()` —
não repetimos código.

Retorno boolean

true → removido com sucesso.

false → matrícula não encontrada.

Listando alunos

```
public void listarAlunos() {  
    for (Aluno alunoAtual : alunos) {  
        System.out.println(  
            alunoAtual.getMatricula() + " - " + alunoAtual.getNome  
            ()  
        );  
    }  
}
```

Saída esperada no console

```
1001 - Maria  
1002 - Joao
```

Boa prática

O main não acessa diretamente o ArrayList. Toda interação passa pelo método listarAlunos().

Exemplo de uso no Main

```
public class Main {  
    public static void main(String[] args) {  
  
        Turma turma = new Turma("LP2");  
  
        Aluno aluno1 = new Aluno(1001, "Maria");  
        Aluno aluno2 = new Aluno(1002, "Joao");  
  
        turma.adicionarAluno(aluno1);  
        turma.adicionarAluno(aluno2);  
  
        turma.listarAlunos();  
  
        boolean ok = turma.removerAlunoPorMatricula(1001);  
        System.out.println("Removido: " + ok);  
    }  
}
```

O main não deve...

...acessar `alunos.add()` diretamente, percorrer a lista ou conhecer a estrutura interna da `Turma`.

Exercício 1: Biblioteca com ArrayList

Crie um sistema simples de biblioteca com as classes abaixo.

Classe Livro (em Livro.java):

- Atributos private: `int isbn` e `String titulo`
- Construtor public recebendo `isbn` e `titulo`
- *Getters* public para cada atributo

Classe Biblioteca (em Biblioteca.java):

- Atributo private `ArrayList<Livro>`
- Inicializar a lista no construtor
- `void adicionarLivro(Livro`

Classe Main (em Main.java):

- Crie uma Biblioteca
- Adicione ao menos 3 livros
- Liste todos os livros
- Busque um livro pelo ISBN e exiba o título
- Remova um livro e liste novamente

Atenção

O main não deve acessar a lista diretamente — use apenas os métodos da classe Biblioteca.

- Representa um **conjunto** de elementos.
- **Não** permite elementos repetidos.
- Não é acessado por posição.
- Garante que um valor apareça apenas uma vez.

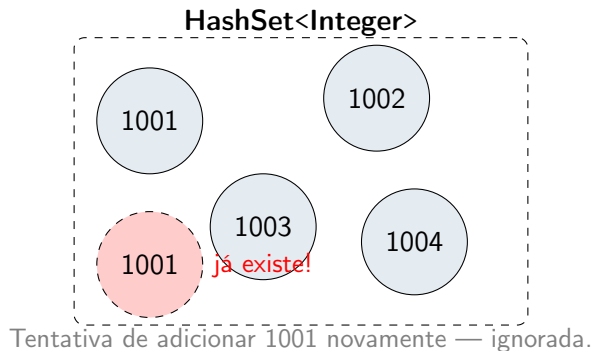
Exemplos de uso

- Matrículas já cadastradas.
- CPFs cadastrados.
- Códigos únicos.
- Palavras sem repetição.

Quando usar?

Use HashSet quando quiser garantir que nenhum elemento se repita no conjunto.

Desenho de um HashSet



Criando um HashSet

```
import java.util.HashSet;

HashSet<String> cpfs = new HashSet<>();

cpfs.add("111.111.111-11");
cpfs.add("222.222.222-22");
cpfs.add("111.111.111-11"); // duplicata ignorada
```

Com ArrayList

Adicionaria o CPF duplicado sem reclamar.

Com HashSet

A duplicata é silenciosamente ignorada — o conjunto permanece com dois elementos.

Métodos básicos do HashSet

```
cpfs.add("111.111.111-11"); // adiciona (ignora se ja existe)
cpfs.contains("111.111.111-11"); // verifica se existe: true/false
cpfs.remove("111.111.111-11"); // remove
cpfs.size(); // quantidade de elementos
cpfs.isEmpty(); // esta vazio? true/false
cpfs.clear(); // remove todos
```

Diferença em relação ao ArrayList

Não existe `get(i)` — o `HashSet` não tem índice.

O retorno do add no HashSet

O método add retorna true ou false — use isso a seu favor.

```
boolean adicionou = cpfs.add("111.111.111-11");

if (adicionou) {
    System.out.println("CPF cadastrado com sucesso.");
} else {
    System.out.println("CPF ja existia no sistema.");
}
```

true

O elemento não existia e foi adicionado.

false

O elemento já existia — nada foi alterado.

Cadastro de CPFs com HashSet

```
import java.util.HashSet;

public class CadastroCPF {

    private HashSet<String> cpfs;

    public CadastroCPF() {
        cpfs = new HashSet<>();
    }

    public boolean cadastrarCPF(String cpf) {
        return cpfs.add(cpf); // false se ja existia
    }

    public boolean cpfJaExiste(String cpf) {
        return cpfs.contains(cpf);
    }

    public boolean removerCPF(String cpf) {
        return cpfs.remove(cpf);
    }
}
```

ArrayList × HashSet

ArrayList	HashSet
Permite repetição	Não permite repetição
Acesso por índice (<code>get(i)</code>)	Sem acesso por índice
Mantém ordem de inserção	Não garante ordem
<code>add</code> sempre insere	<code>add</code> retorna <code>false</code> se já existe
Bom para listar em sequência	Bom para garantir unicidade
<i>Ex: alunos de uma turma</i>	<i>Ex: matrículas já usadas</i>

Exercício 2: Controle de presença

Crie um sistema de controle de presença para um evento diferente da aula — por exemplo, um **festival de cinema**.

Classe ControleIngresso:

- private HashSet<String> de códigos de ingressos validados
- Construtor inicializando o HashSet
- boolean validarIngresso(String codigo)
- boolean ingressoJaUsado(String codigo)
- boolean cancelarIngresso(String

No Main:

- Valide ao menos 3 ingressos
- Tente validar um ingresso já usado e exiba a mensagem adequada
- Cancele um ingresso e confirme

Regra

O mesmo código de ingresso não pode ser validado duas vezes.

- Guarda pares de **chave** → **valor**.
- Cada chave aponta para um valor.
- A **chave não se repete**.
- Busca **instantânea** pela chave — sem percorrer tudo.

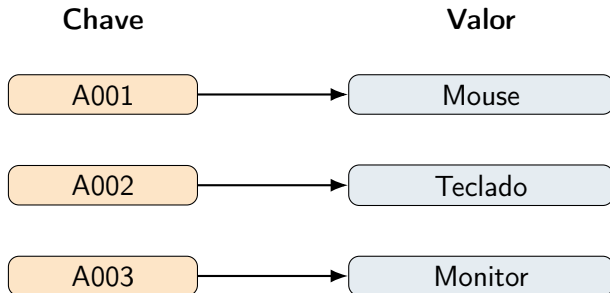
Sintaxe

```
HashMap<TipoChave, TipoValor>
```

Exemplos

- CPF → Aluno
- Código → Produto
- ISBN → Livro
- Matrícula → Nota

Desenho de um HashMap



Cada chave é única e aponta para exatamente um valor.

Criando um HashMap

```
import java.util.HashMap;

HashMap<String, String> produtos = new HashMap<>();

produtos.put("A001", "Mouse");
produtos.put("A002", "Teclado");
produtos.put("A003", "Monitor");

System.out.println(produtos.get("A002")); // Teclado
```

Primeiro tipo

Tipo da **chave** (String aqui).

Segundo tipo

Tipo do **valor** (String aqui).

put e get

put insere; get busca pela chave.

Métodos básicos do HashMap

```
produtos.put("A001", "Mouse");           // adiciona ou atualiza
produtos.get("A001");                     // busca pelo chave -> "Mouse"
produtos.containsKey("A001");             // chave existe? true/false
produtos.containsValue("Mouse");         // valor existe? true/false
produtos.remove("A001");                  // remove pela chave
produtos.size();                          // quantidade de pares
produtos.isEmpty();                       // esta vazio? true/false
produtos.clear();                         // remove todos
```

Cuidado com put

Se a chave já existir, o valor anterior é **substituído** silenciosamente.

Classe Produto encapsulada

```
public class Produto {  
  
    private String codigo;  
    private String nome;  
    private double preco;  
  
    public Produto(String codigo, String nome, double preco) {  
        this.codigo = codigo;  
        this.nome = nome;  
        if (preco >= 0) {  
            this.preco = preco;  
        }  
    }  
  
    public String getCodigo() { return codigo; }  
    public String getNome() { return nome; }  
    public double getPreco() { return preco; }  
}
```

Lembre-se

Salvar em Produto.java. O preço só é aceito se for não-negativo.

Estoque com HashMap

```
import java.util.HashMap;

public class Estoque {

    private HashMap<String, Produto> produtosPorCodigo;

    public Estoque() {
        produtosPorCodigo = new HashMap<>();
    }

    public boolean cadastrarProduto(Produto produto) {
        String codigo = produto.getCodigo();
        if (produtosPorCodigo.containsKey(codigo)) {
            return false; // ja existe, nao substitui
        }
        produtosPorCodigo.put(codigo, produto);
        return true;
    }

    public Produto buscarProduto(String codigo) {
        return produtosPorCodigo.get(codigo);
    }
}
```

Removendo produto pelo código

```
public boolean removerProduto(String codigo) {  
    Produto removido = produtosPorCodigo.remove(codigo);  
    return removido != null;  
}
```

remove encontrou

Retorna o objeto removido →
diferente de null → retornamos
true.

remove não encontrou

Retorna null → chave não existia
→ retornamos false.

Percorrendo um HashMap

Pelos **valores**:

```
public void listarProdutos() {  
    for (Produto p : produtosPorCodigo.values()) {  
        System.out.println(p.getCodigo() + " - " + p.getNome());  
    }  
}
```

Pelas **chaves**:

```
public void listarCodigos() {  
    for (String codigo : produtosPorCodigo.keySet()) {  
        System.out.println(codigo);  
    }  
}
```

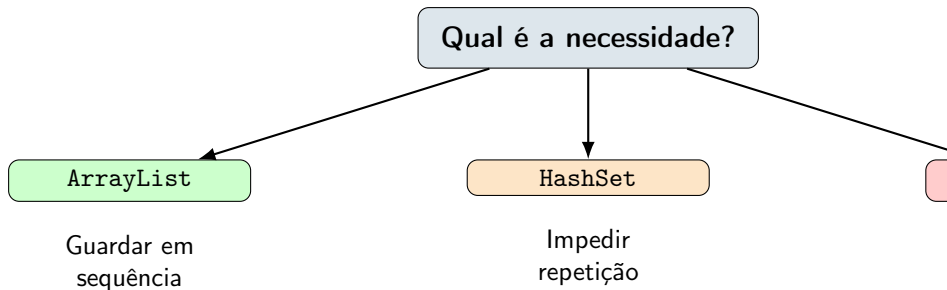
Resumo

`values()` → os valores. `keySet()` → as chaves.

ArrayList × HashSet × HashMap

Característica	ArrayList	HashSet	HashMap
Organização	Lista	Conjunto	Chave e valor
Permite repetição	Sim	Não	Chave não repete
Acesso por índice	Sim	Não	Não
Busca por chave	Não diretamente	Não diretamente	Sim (get)
add/put	Sempre insere	Retorna false se repetido	Substitui se chave existir
Exemplo	Lista de alunos	CPFs únicos	Produto por código

Como escolher a coleção?



Erro comum: não inicializar a coleção

Errado

```
1 public class Turma {
2
3     private ArrayList<Aluno>
4         alunos;
5
6     public void adicionarAluno(
7         Aluno aluno) {
8         alunos.add(aluno);
9         // NullPointerException!
10    }
11 }
12
13
```

Correto

```
1 public class Turma {
2
3     private ArrayList<Aluno>
4         alunos;
5
6     public Turma() {
7         alunos = new ArrayList
8             <>();
9     }
10
11     public void adicionarAluno(
12         Aluno aluno) {
13         alunos.add(aluno);
14     }
15 }
```

Regra

Erros de estrutura

- Esquecer o `import` da coleção.
- Declarar o atributo mas não inicializar no construtor.
- Deixar a coleção como `public`.
- Construtor sem `public`.

Erros de lógica

- Manipular a coleção diretamente no `main`.
- Usar `ArrayList` quando precisava de unicidade.
- Usar `HashMap` sem verificar se a chave já existe antes do `put`.

Exercício 3: Locadora de filmes

Crie um sistema de locadora usando HashMap.

Classe Filme (Filme.java):

- `private String codigo`
- `private String titulo`
- `private String diretor`
- Construtor e *getters* públicos

Classe Locadora (Locadora.java):

- `private HashMap<String, Filme>`
- `boolean cadastrarFilme(Filme f)`
- `Filme buscarFilme(String codigo)`

No Main:

- Cadastre ao menos 3 filmes
- Tente cadastrar um código repetido e exiba aviso
- Busque um filme pelo código e exiba título e diretor
- Remova um filme e liste os restantes

Coleção

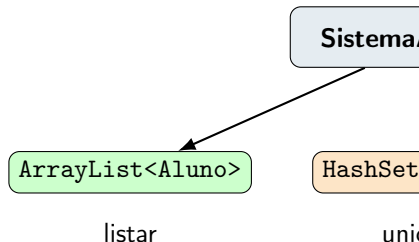
Use `HashMap<String, Filme>` — o código é a chave.

Exercício 4: Sistema acadêmico

Crie a classe `SistemaAcademico` usando as **três coleções** juntas:

- `ArrayList<Aluno>` — lista de alunos em ordem de cadastro
- `HashSet<Integer>` — matrículas já usadas (impede repetição)
- `HashMap<Integer, Aluno>` — busca rápida por matrícula

Métodos:



Desafio

Ao cadastrar, insira nas três coleções. A matrícula não pode se repetir.

Estrutura inicial do SistemaAcademico

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.HashMap;

public class SistemaAcademico {

    private ArrayList<Aluno>      alunos;
    private HashSet<Integer>      matriculasUsadas;
    private HashMap<Integer, Aluno> alunosPorMatricula;

    public SistemaAcademico() {
        alunos      = new ArrayList<>();
        matriculasUsadas = new HashSet<>();
        alunosPorMatricula = new HashMap<>();
    }
}
```

Cadastrando aluno sem repetir matrícula

```
public boolean cadastrarAluno(Aluno aluno) {
    int matricula = aluno.getMatricula();

    if (matriculasUsadas.contains(matricula)) {
        return false; // matricula ja existe
    }

    alunos.add(aluno); // ArrayList
    matriculasUsadas.add(matricula); // HashSet
    alunosPorMatricula.put(matricula, aluno); // HashMap

    return true;
}
```

Por que três coleções?

Cada uma tem um papel: o HashSet garante unicidade, o HashMap permite busca rápida e o ArrayList mantém a ordem de cadastro.

Buscando aluno no HashMap

```
public Aluno buscarAlunoPorMatricula(int matricula) {  
    return alunosPorMatricula.get(matricula);  
}
```

Com ArrayList

Seria necessário percorrer toda a lista comparando cada matrícula — mais lento.

Com HashMap

Uma única operação `get` localiza o aluno diretamente pela chave.

As três coleções

- `ArrayList` → lista ordenada
- `HashSet` → sem repetição
- `HashMap` → chave e valor

Boas práticas

- Classes e construtores `public`
- Atributos `private`
- Coleção inicializada no construtor
- Acesso apenas por métodos públicos

Checklist rápido

- ✓ Importei a coleção correta?
- ✓ Atributo está `private`?
- ✓ Iniciei no construtor?
- ✓ Criei métodos para `add`, `buscar` e `remover`?
- ✓ Valido duplicatas quando necessário?
- ✓ O `main` usa só os métodos públicos?
- ✓ Escolhi a coleção certa para o problema?